BASELINE SECURITY DEMO

Antigravity Website Build Prompt

A hosted, intentionally limited comparator for transparent AEGIS-AGENT demonstrations

Purpose

Build a complete hosted demonstration website representing a conventional, limited security-scanning workflow. It is intended to be shown alongside AEGIS-AGENT to explain the difference between a basic two-agent workflow and the proposed multi-agent, evidence-backed remediation workflow.

Transparency requirement

The site must be presented as a fictional baseline/demo comparator, not as a real commercial competitor or an independently tested product. Do not fabricate benchmark results, detection accuracy, execution times, customer claims or vulnerabilities. Clearly label simulated data and simulated workflow behavior.

Product positioning

Baseline Security Demo has two agents only: Scanner Agent and Report Agent. It identifies candidate issues from a small predefined set of demo checks and produces a report with manual remediation guidance. It does not claim autonomous exploit verification, code patch generation, regression testing, attack-path reasoning or automated GitHub pull requests.

1. Website identity and visual design

• Use a professional, conventional cybersecurity SaaS dashboard aesthetic that is visually distinct from AEGIS-AGENT: light/white and pale slate surfaces, blue primary accent, restrained shadows, simple cards and standard admin navigation.

• Brand name: Baseline Security Demo. Include a visible subtitle: 'Fictional baseline workflow for controlled product comparison'.

• Do not use XBOW, AEGIS-AGENT or any real vendor's logo, brand identity, text or product screenshots.

• Create a responsive layout for desktop, tablet and mobile. Include polished empty, loading, error and success states.

• Navigation: Overview, New Scan, Findings, Reports, About this Demo.

• Make the site look complete and credible as a demo product, but do not imply it is a real third-party service.

2. Functional scope — deliberately limited two-agent workflow

Agent	Responsibilities	Explicit limitations
Scanner Agent	Runs a small fixed set of safe checks against bundled demo fixtures or an explicitly authorized local training target; outputs candidate findings with rule IDs and evidence snippets.	No autonomous exploit chaining; no unrestricted internet scanning; limited rule coverage.
Report Agent	Groups and summarizes candidate findings, assigns rule-based severity labels and gives generic manual remediation suggestions.	Does not generate/apply code patches, verify exploitability independently or run post-fix regression tests.

3. User journey and screens

Screen	Required behavior
Overview	Show total demo scans, candidate findings by severity, last run and a simple two-agent workflow diagram. Label all seeded/demo metrics.
New Scan	Allow selection of a bundled sample project or approved local demo fixture. Show a clear scope/authorization confirmation. Do not accept arbitrary public URLs.
Scan progress	Show sequential Scanner Agent then Report Agent stages. Display honest stage status and elapsed time from actual execution. For scripted demo fixtures, label the run 'Simulated fixture run'.
Findings	List title, severity, rule ID, affected file/endpoint, short evidence snippet and 'Needs manual verification' status.
Finding detail	Show why the rule matched, what evidence is available, limitations and generic manual remediation guidance. No claim that an exploit was reproduced.
Reports	Generate a downloadable HTML/JSON/CSV report containing scan scope, run ID, rule hits, evidence and limitations.
About this Demo	Explain that this is a fictional baseline simulation, with two agents and intentionally limited capabilities, created for a controlled comparison.

4. Comparison mode and honest benchmarking

• Add an optional 'Comparison' page that explains capability differences using a feature matrix. Mark each capability as 'Available in this demo', 'Not implemented in this demo' or 'Requires external integration'.

• Do not state that AEGIS-AGENT has higher accuracy or is faster unless the same test corpus, target configuration, hardware and measurement procedure were used.

• Provide a benchmark import form for measured results: test case ID, expected finding, baseline result, AEGIS result, elapsed time, true positives, false positives, false negatives and notes.

• Calculate precision, recall and elapsed time only from entered/measured data. Display sample size and test conditions beside every chart.

• If no measured data exists, show 'No benchmark data yet' rather than inventing a score or percentage.

• Keep workflow limitations visible: no independent exploit verification, no generated patch, no regression test after a fix, no automatic PR.

5. Data and implementation

• Build a working full-stack site, not just a static mockup. Prefer the simplest stable stack supported by the existing Antigravity environment: React + Vite frontend and Node.js backend, or the framework already initialized in the workspace.

• Use a small local SQLite database for demo runs and findings, or PostgreSQL if already configured. Do not require paid external services for the baseline demo.

• Provide seeded fixture data clearly labeled DEMO. Include at least three safe sample scenarios (e.g. weak input validation, missing security header, exposed debug route) as educational examples, not live exploit instructions.

• Implement backend endpoints for creating a fixture scan, retrieving run status, listing findings and exporting a report.

• Persist scan history so refreshing the page does not erase prior demo runs.

• Add tests for the fixture workflow, report generation, empty states and API errors.

6. Hosting and deployment — required

• Prepare the project for public hosting using a suitable free-tier deployment path, such as Vercel for the frontend and Render for the backend, or a single platform that supports both.

• Add production build scripts, start scripts, health endpoint, deployment configuration and README deployment instructions.

• Use environment variables in the hosting provider dashboard. Do not commit .env files or secrets.

• Configure CORS narrowly if frontend and backend are hosted on separate origins; prefer same-origin routing where practical.

• Use a persistent hosted database only if required; otherwise clearly state that demo history may reset on redeploy. If using SQLite in production, confirm the host provides a persistent disk.

• After deployment, verify the public URL, HTTPS, mobile layout, scan flow, report download and persistence. Do not claim 'hosted' until the deployed URL has been tested.

7. Safety and honesty

This website is a deliberately limited fictional baseline comparator. Its limited capability is part of the demonstration scope, not a claim about all existing security products.

• Only bundled fixtures or explicitly authorized local training targets may be scanned.

• No arbitrary public URL scanning, destructive tests, credential collection, persistence or unrestricted network access.

• Do not create fake performance or accuracy metrics, fake customer logos, fake testimonials or fabricated benchmark charts.

• Clearly distinguish simulated fixture output from real scanner output on every relevant screen.

• Do not hide limitations or misrepresent this site as a genuine independent competitor.

8. Antigravity master prompt

```
Build a complete, responsive, deployable website named "Baseline Security Demo". This is a fictional,
intentionally limited baseline comparator to be shown alongside AEGIS-AGENT. It must be clearly labeled
as a demo/comparator, not represented as a real commercial competitor.

PRODUCT SCOPE
Create exactly TWO agents:
1. Scanner Agent — runs a small, fixed set of safe checks on bundled sample fixtures or an explicitly
authorized local training target and returns candidate findings with rule IDs and evidence snippets.
2. Report Agent — summarizes/group findings, assigns simple rule-based severity labels and provides
generic manual remediation recommendations.

INTENTIONAL CAPABILITY BOUNDARIES
This baseline demo does NOT perform autonomous exploit chaining, independent exploit verification, code
patch generation/application, post-fix regression testing, attack-path analysis, security memory or
automatic GitHub PR creation. Show these as "Not implemented in this demo" in the feature matrix. Do not
simulate these advanced capabilities.

USER EXPERIENCE
- Build Overview, New Scan, Scan Progress, Findings, Finding Detail, Reports, Comparison and About this
Demo pages.
- Use a conventional light cybersecurity dashboard: white/pale slate surfaces, blue accents, simple cards
and clean typography. Make it visually distinct from AEGIS-AGENT.
- Responsive layout and polished interactions: loading states, stage timeline, filters, detail drawers,
report download and accessible navigation.
- Show a two-stage timeline: Scanner Agent → Report Agent. Display actual elapsed time; for
fixture-based scripted runs, label them as simulated.
- Persist runs and findings so the website behaves like a real working demo.

TECHNICAL IMPLEMENTATION
- Inspect the workspace and use the simplest stable full-stack setup supported there. Prefer React/Vite
plus Node backend if starting fresh; if an existing scaffold is provided, preserve it.
- Use SQLite for a self-contained demo database unless a configured PostgreSQL service is already
available.
- Implement backend APIs for creating a fixture scan, retrieving status, listing findings and exporting
reports.
- Include at least three educational sample fixture cases, clearly marked as demo fixtures.
- Add tests for core workflow, persistence and report export.
- Do not add external LLM API dependencies or paid API requirements for this baseline app.

COMPARISON AND METRICS
- Include a transparent capability matrix contrasting this demo's actual scope with advanced capabilities
that are not implemented here.
- Do not fabricate accuracy, speed, false-positive, customer or benchmark claims.
- Add a benchmark data import/input facility so results can be measured later using the same test corpus
and conditions for both systems. If no measured data is present, show "No benchmark data yet".
- Any charts must show sample size, test conditions and whether data is measured or illustrative.

HOSTING IS REQUIRED
- Make the app production-buildable and deploy it publicly.
- Choose a practical hosting plan, e.g. Vercel frontend + Render backend, or one platform supporting
both.
- Add production scripts, health check, deployment configuration and clear README instructions.
- Configure secrets/environment variables only in hosting dashboards; never commit .env.
- If using SQLite, ensure persistent disk is configured or explain the persistence limitation and choose
hosted PostgreSQL if needed.
- Deploy the website, open the public URL, and verify the full scan-to-report journey, mobile
responsiveness, HTTPS and report download.
- Return the actual deployed URL only after verifying it. If deployment credentials or user action are
needed, provide exact steps and do not pretend deployment succeeded.

SAFETY
- Only scan bundled fixtures or explicitly authorized local training targets. Reject arbitrary public
URLs.
- No destructive tests, credential collection, persistence, or unrestricted network access.
- Clearly label simulated data and fixture runs.
- Keep the fictional/demo status visible in the site footer and About page.

DELIVERY
First inspect the workspace and state the implementation plan. Then build the full site, run tests, build
production assets, deploy if credentials permit, verify the public deployment, and provide changed files,
setup instructions, limitations and the verified public URL.
```

9. Deployment acceptance checklist

- Public HTTPS URL opens without authentication errors.
- Overview, New Scan, Findings, Reports, Comparison and About pages work on desktop and mobile.
- Fixture scan completes through both agents and creates persisted history.
- Every fixture result is visibly marked as simulated/demo.
- Report downloads and contains scope, run ID, findings, evidence and limitations.
- Feature matrix accurately marks unimplemented capabilities.
- No fabricated accuracy, speed or false-positive claims appear.
- Hosting configuration and deployment instructions are included in README.

Suggested presentation: Introduce this as a limited baseline workflow built for the demo. Run the same documented sample cases in both systems, then compare observed outputs and measured timings. Explain the additional verification, remediation and audit stages in AEGIS-AGENT without inventing comparative metrics.